Task 7: Integration and Demonstration

TaskForge Demo Script

Opening

TaskForge manages a software delivery workflow. Work is organized into sprints, which can contain both individual tasks and nested sub-groups. Individual tasks move through a lifecycle, from Design through to Deployed, and can pick up extra responsibilities at runtime, like being flagged urgent, without needing a new subclass for every combination.

Scenario 1: Fast-tracking a feature

Run it live, narrate as it prints:

Here's Sprint 12, with two tasks. First I traverse it with a TGIterator, which visits every task without me ever touching the group's internal vector directly, that's the Iterator pattern keeping traversal separate from the group's actual storage.

Now Fix login bug gets flagged urgent. I don't subclass UnitTask, I wrap it in a PriorityDecorator. That's a structural change at runtime, I remove the plain task from the group and add back the decorated version, ownership transfers cleanly: the decorator now owns the task, the group owns the decorator.

Then I advance its state while it's still sitting inside the group. Because the decorator forwards updateState() to whatever it wraps, the real underlying task's lifecycle actually changes. Re-traversing the group shows both the priority label and the new state together.

Scenario 2: A rejected review

Sprint 13 has a task already sitting in Under Review. Here I use a different iterator, StateIterator, which filters by lifecycle state instead of visiting everything. That's the second, meaningfully different traversal the spec requires.

The reviewer rejects it. UnderReview has a specific reject path that sends the task back to Implementation, notice this doesn't require the same test-passed condition as approving forward, since sending something back for rework isn't the same kind of transition as approving it.

Filtering again with StateIterator, now for Implementation, finds the same task under its new state, that's a runtime change, but a regression, not just forward progress.

Traversal-invalidation policy

We settled on a snapshot policy, an iterator should be unaffected by structural changes made after it's created. Right now that's the intended design, not yet fully enforced, since neither iterator independently copies the children vector, that's a known gap we're flagging rather than hiding.

Ownership and destruction

TaskGroup owns and deletes its children in its destructor. TaskDecorator owns and deletes whatever it wraps. TaskGroup::remove(), which we added specifically to support Scenario 1, deliberately does not delete, ownership transfers to the caller instead, that's what lets a plain task survive being pulled out and re-wrapped.

GDB / Valgrind

We hit a real double-delete bug during development, calling delete this inside a state's updateState() after setState() had already deleted the same object. AddressSanitizer caught it as a heap-use-after-free. Final build runs clean under Valgrind, no leaks.

Closing

That's the four patterns working together as one system, not four separate demos, plus a documented, if not fully enforced, policy for what happens when the structure changes underneath a live traversal.